

1: INTRODUCTION

In the contemporary landscape of education, the paradigm is shifting from passive information consumption to active, participatory learning. As digital literacy becomes a prerequisite for academic and professional success, educators and institutions require dynamic tools to assess knowledge, encourage healthy competition, and make learning engaging. Traditional examination methods—often characterized by manual grading, delayed feedback, and high-stress environments—can fail to motivate students or provide the instant gratification required to maintain interest in a subject.

The rapid democratization of the internet has enabled new avenues for interactive learning. However, while social media platforms have transformed how information is shared, they often lack a focused, academic-centric environment free from distractions. Furthermore, many existing e-learning tools lack the "gamified" elements—such as real-time leaderboards and competitive scoring—that drive modern user engagement.

"Quiz by Shivam" is designed to bridge this gap. It is a comprehensive, feature-rich web application built on the **WordPress** Content Management System (CMS). Unlike static educational websites, this platform offers a streamlined approach to collaborative learning through gamification. It provides a dedicated environment where students can test their proficiency across various domains, including Technology, Business, Science, and General Knowledge.

The core philosophy of this project is to create a "one-stop" assessment solution that makes learning competitive and accessible. By leveraging modern web technologies (PHP, MySQL, and JavaScript), the platform provides a seamless quiz-taking experience. It replaces the monotony of standard tests with a vibrant interface featuring timed challenges, instant score calculation, and a transparent global

leaderboard. This competitive approach utilizes the psychological principles of gamification to motivate users to improve their scores and revisit categories where they previously underperformed.

From a technical perspective, the platform is rooted in full-stack web development principles. By utilizing WordPress, the project benefits from a modular architecture that ensures scalability, security, and ease of administration. The application follows an iterative development approach, ensuring that critical features—such as user authentication, quiz logic, and database management—are refined continuously to meet the needs of modern students and enthusiasts.

2. Objectives of the Project

The primary aim of this project is to design and develop a responsive, dynamic quiz application that facilitates automated assessment and user ranking. The specific objectives are as follows:

- **To Develop a Robust Assessment Engine:** To create a system capable of handling various quiz formats (multiple choice, true/false) with features such as countdown timers, automated grading, and instant result display.
- **To Implement Gamification Strategy:** To motivate continuous learning by integrating a real-time global leaderboard that ranks users based on their cumulative scores, fostering a spirit of healthy competition.
- **To Categorize Academic Content:** To facilitate easy navigation through a structured, category-based repository, allowing users to select specific subjects of interest (e.g., Web Technologies, DBMS, Artificial Intelligence) for focused revision.
- **To Personalize the User Experience:** To provide a dedicated user dashboard where students can manage their profiles, track their quiz history, view past performance statistics, and update personal details.
- **To Ensure Cross-Platform Accessibility:** To design a responsive, mobile-friendly interface using HTML5, CSS3, and WordPress themes, ensuring the platform functions seamlessly across desktops, tablets, and smartphones.
- **To Streamline Administration:** To provide a secure backend interface for administrators to easily create, edit, and delete questions without requiring direct code manipulation.

Scope of the Project

The scope of "Quiz by Shivam" defines the boundaries of the system, outlining the specific modules and functionalities that will be delivered. The project is divided into the following key modules:

1. User Management Module:

- **Authentication:** Secure registration and login systems to protect user identity.
- **Profile Management:** Capability for users to customize their profiles (avatars, bios) and view their own statistical data, such as total quizzes attempted and average score.

2. The Quiz Engine (Core Functionality):

- **Timed Assessments:** Implementation of logic to handle time-restricted questions to simulate exam conditions.
- **Instant Feedback:** Algorithms to calculate scores immediately upon submission and provide correct answers for review.
- **Category Filtering:** A dynamic sorting system allowing users to filter quizzes by tags (e.g., "Programming," "Science") or difficulty levels.

3. The Gamification & Ranking Module:

- **Global Leaderboard:** A dynamic table that queries the database to display the top-performing users.
- **Score Tracking:** Persistent storage of user scores in the MySQL database to track progress over time.

4. Administrative Control Panel:

- **Content Management:** A dashboard allowing the admin to add new questions, manage categories, and moderate user activity.
- **System Maintenance:** Tools to ensure data integrity and manage site settings.

5. Technical Scope:

- **Frontend:** HTML, CSS, and JavaScript for a responsive and interactive UI.
- **Backend:** PHP (WordPress Core) for server-side logic and session management.
- **Database:** MySQL for storing user credentials, questions, answers, and score logs.

3(A) INTRODUCTION TO HTML AND CSS

HTML (Hypertext Markup Language) and CSS (Cascading Style Sheets) serve as the foundational technologies for the presentation layer of "Quiz by Shivam." Within the WordPress framework, these technologies work in tandem to construct a dynamic, visually engaging, and fully responsive user interface.

1. HTML (Hypertext Markup Language) HTML functions as the structural backbone of the application. In the WordPress ecosystem, HTML is dynamically generated via **PHP templates**. It defines the semantic layout of critical components such as the Navigation Bar, Quiz Category Cards, Question Containers, and the Leaderboard Table.

- **Semantic Structure:** The project utilizes semantic HTML5 elements (e.g., `<header>`, `<main>`, `<article>`, `<section>`) to categorize content effectively.
- **Accessibility & SEO:** By using proper tags for forms, buttons, and navigation links, the structure ensures the platform is accessible to screen readers and optimized for Search Engine engines (SEO).

2. CSS (Cascading Style Sheets) & Frameworks CSS is responsible for the visual presentation and aesthetic consistency of the platform. It manages the layout, color palette, typography, and spacing.

- **Custom Styling:** The project utilizes a dedicated stylesheet (e.g., `style.css` or `quiz-custom.css`) to define the unique visual identity of the application. This includes specific styling for the quiz timer, the hover effects on answer options, and the highlighting of correct/incorrect answers.

- **Bootstrap Framework:** To accelerate interface development and ensure cross-device compatibility, the project integrates the **Bootstrap** framework. This provides a robust grid system that automatically adjusts the layout based on the user's screen size, ensuring a seamless experience whether the quiz is taken on a desktop, tablet, or smartphone.

3(B) INTRODUCTION TO JAVASCRIPT (CLIENT-SIDE LOGIC)

JavaScript is the engine that drives the interactivity of the "Quiz by Shivam" platform. While WordPress handles the server-side data processing via PHP, JavaScript executes on the client side (the user's browser) to create a reactive and engaging experience without requiring frequent page reloads.

Key Roles of JavaScript in the Project:

1. Quiz Interaction & Timer Logic JavaScript is critical for the integrity of the assessment process.

- **Countdown Timer:** A JavaScript-based timer runs during active quiz sessions. It tracks the remaining time and triggers an "Auto-Submit" function when the clock hits zero, ensuring fair, timed assessments for all participants.
- **Dynamic UI Updates:** JavaScript handles DOM (Document Object Model) manipulation to update the interface instantly. This includes disabling the "Submit" button once clicked to prevent duplicate submissions and highlighting selected answers for better user feedback.

2. Asynchronous Operations (AJAX) To provide a modern, app-like experience, the project utilizes **AJAX** (Asynchronous JavaScript and XML).

- **Seamless Data Submission:** When a user submits a quiz, JavaScript captures the form data and sends it to the WordPress backend asynchronously. This allows the system to calculate scores and return the results instantly without refreshing the entire web page.
- **Real-Time Feedback:** AJAX is used to fetch feedback on answers immediately, improving the learning curve by showing users where they went wrong in real-time.

3. Event Handling The application relies on JavaScript event listeners to respond to user actions. This includes validating that an option has been selected before moving to the next question, handling pagination between quiz sections, and managing modal pop-ups for the registration and login forms.

3(C) INTRODUCTION TO PHP AND WORDPRESS

PHP (Hypertext Preprocessor) is a widely used, open-source, server-side scripting language designed specifically for web development. **WordPress**, the world's most popular Content Management System (CMS), is architected entirely upon PHP. For the "Quiz by Shivam" project, PHP serves as the logic layer that processes user requests, calculates scores, and dynamically generates the HTML views displayed to the user.

The Role of PHP in "Quiz by Shivam": Unlike client-side languages that run in the browser, PHP executes on the web server (hosted at `shivam.24x7ithub.com`). When a student attempts a quiz, PHP handles the critical logic: it retrieves questions from the database, validates the submitted answers against the answer key, calculates the final score, and updates the user's profile—all securely on the server before rendering the result page.

Why PHP & WordPress were selected: While traditional web development requires building every feature from scratch, WordPress extends PHP with a robust ecosystem. It provides a structured **Plugin Architecture** and **Theme System**, allowing for rapid development. This architecture is crucial for this project as it streamlines complex tasks such as User Authentication (Login/Register) and Role Management (Admin vs. Student).

Key Technical Features of the Stack:

- **Server-Side Rendering (SSR):** PHP generates HTML dynamically on the server. This ensures that the quiz content is SEO-friendly and compatible with all browsers, as the heavy lifting is done by the server, not the user's device.
- **The WordPress Hook System (Actions & Filters):** This is the core of WordPress extensibility. The project uses "Hooks" to intercept data at specific points. For example, when a

user finishes a quiz, a custom PHP function hooks into the submission process to immediately save the score to the Leaderboard table.

- **REST API Integration:** WordPress includes a built-in REST API. This allows the frontend (JavaScript) to communicate with the backend (PHP) using JSON. This is utilized in the "Quiz by Shivam" for features like infinite scroll or fetching quiz questions without reloading the page.
- **Session Management:** PHP handles user sessions securely. It ensures that when a user logs in, their identity is preserved across different pages, allowing them to track their history and maintain a persistent profile.
- **Scalability & Performance:** The application is hosted on a Linux-based environment. PHP 8.x (the latest version) utilizes the **Zend Engine** and **OPCache**, which compiles scripts into bytecode and stores them in memory. This reduces the loading time significantly, ensuring that the platform can handle multiple students attempting quizzes simultaneously without latency.

3(D) INTRODUCTION TO MYSQL AND WORDPRESS DATABASE

MySQL is an open-source Relational Database Management System (RDBMS) based on **SQL (Structured Query Language)**. It is the standard database service for WordPress. Unlike NoSQL databases that store unstructured data, MySQL stores data in highly structured **Tables** consisting of **Rows** and **Columns**.

Data Structure in "Quiz by Shivam": The database acts as the memory of the application. The validity and integrity of the quiz competition rely on the structured nature of MySQL. The project utilizes the core WordPress tables alongside custom implementations:

1. **wp_users:** Stores authentication details (Username, Password Hash, Email).
2. **wp_posts:** Stores the quiz questions and supporting content.
3. **wp_postmeta:** Stores metadata, such as the correct answer for a specific question or the time limit for a specific quiz.
4. **Custom Leaderboard Table:** A dedicated table created to store `User_ID`, `Quiz_Category`, and `Score`, optimized for fast retrieval during ranking.

Key Database Concepts Implemented:

- **Relational Integrity:** MySQL enforces relationships between data points. For example, a "Score" entry is strictly linked to a specific "User ID." If a user is deleted, their associated scores can also be managed automatically, ensuring data consistency.
- **CRUD Operations:** The project relies on the four fundamental database operations, managed via the global `$wpdb` class in WordPress:
 - **Create (INSERT):** Used when a new user registers or a new score is recorded.

- **Read (SELECT):** Used to fetch questions for the quiz or retrieve the top 10 scores for the leaderboard.
- **Update (UPDATE):** Used when a user edits their profile bio or an admin corrects a typo in a question.
- **Delete (DELETE):** Used by admins to remove inappropriate comments or outdated quizzes.
- **ACID Compliance:** MySQL follows ACID properties (Atomicity, Consistency, Isolation, Durability). This is critical for a competition platform; it ensures that if two students submit a quiz at the exact same millisecond, both scores are recorded accurately without data collision or corruption.
- **Indexing for Performance:** To ensure the Leaderboard loads instantly, the database uses **Indexes** on columns like `score` and `category`. An index acts like a table of contents, allowing the database to find the top scores without scanning the entire history of every quiz ever taken.

Integration with WordPress: The interaction between PHP and MySQL is abstracted through the **WordPress Database Class (\$wpdb)**. This provides a secure method to interact with the database, automatically protecting the application against **SQL Injection attacks** (a common web security threat) by sanitizing all data before it enters the database.

4. DEFINITION OF PROBLEM

The current landscape of online learning and knowledge assessment faces several significant challenges related to user engagement, motivation, and accessibility. While students utilize various digital tools for studying, they often lack a centralized, competitive, and structured platform dedicated to testing their knowledge.

The primary problems that "Quiz by Shivam" addresses are as follows:

1. Lack of Interactive Assessment Tools

- **Issue:** Traditional academic assessments and many existing online quizzes are static and text-heavy. They lack "gamification" elements such as timers, instant feedback, and visual rewards.
- **Impact:** This leads to a passive learning experience where students quickly lose interest, reducing the effectiveness of self-assessment.

2. Fragmentation of Academic Content

- **Issue:** Students currently have to visit multiple disparate websites to practice questions for different subjects (e.g., one site for DBMS, another for Networking, and a third for General Knowledge).
- **Impact:** This fragmentation wastes time and makes it difficult for students to have a holistic revision strategy. A unified, category-based system is required to bring all subjects under one roof.

3. Absence of Competitive Motivation (Leaderboards)

- **Issue:** Most e-learning platforms offer solitary experiences. Students take a quiz, see a score, and leave. There is no mechanism to compare performance with peers.
- **Impact:** Without a sense of competition or ranking, students lack the intrinsic motivation to improve their scores. The absence of a "Global Leaderboard" results in lower retention and engagement rates.

4. Poor Mobile Accessibility

- **Issue:** Many institutional educational portals are designed for desktop use, offering poor user experiences on smartphones.
- **Impact:** Since the majority of students access the internet via mobile devices, a non-responsive interface alienates a large portion of the user base.

5. Inefficiency in Manual Evaluation

- **Issue:** Manual grading of quizzes or managing physical question papers is time-consuming and prone to human error.
- **Impact:** Delayed results prevent students from identifying their weak areas immediately. An automated system is needed to provide instant scoring and result analysis.

6. Data Security and User Identity

- **Issue:** Basic HTML quizzes often lack secure authentication, allowing users to manipulate results or take quizzes anonymously without tracking progress.
- **Impact:** The system requires robust authentication (User Registration/Login) to ensure that progress is saved, and result integrity is maintained.

5. SYSTEM ANALYSIS

System analysis for "Quiz by Shivam" involves a detailed examination of the requirements of students and administrators. The goal is to design a system that provides a seamless quiz-taking experience while ensuring data security, efficient database management, and accurate leaderboard calculations.

6.1 Problem Identification

The analysis phase identified three core gaps in the existing environment:

- **Functionality Gap:** Lack of a platform that combines timed quizzes with instant result generation.
- **Content Gap:** The need for a single repository where questions are organized by specific technical categories (DBMS, OS, AI).
- **Social Gap:** The lack of a leaderboard system to foster a competitive academic community.

6.2 User Requirements

A. The Student (User)

- **Registration:** Must be able to create a secure account to save progress.
- **Browse & Search:** Needs an interface to easily select quizzes based on categories (e.g., "Science," "Technology").
- **Quiz Interface:** Requires a distraction-free interface with a countdown timer and clear options.
- **Feedback:** Expects instant scoring and an overview of correct vs. incorrect answers upon completion.
- **Ranking:** Wants to see their name on the Global Leaderboard if they score well.

B. The Administrator

- **Dashboard:** Requires a WordPress Admin Panel to manage the site without writing code.
- **CRUD Operations:** Needs the ability to Create, Read, Update, and Delete questions and categories.
- **User Management:** Ability to monitor registered users and remove spam accounts.

6.3 System Requirements

Functional Requirements:

1. **User Authentication:** Secure Login/Register functionality using WordPress Core sessions.
2. **Quiz Engine:** A logic module that handles question rotation, timer countdowns, and answer validation.
3. **Scoring Algorithm:** Automated calculation of scores based on correct answers (e.g., +10 points per correct answer).
4. **Leaderboard System:** A dynamic display table that queries the database to rank users by total score.
5. **Responsive Design:** The UI must adapt to Desktop, Tablet, and Mobile screens.

Non-Functional Requirements:

1. **Scalability:** The database structure must support thousands of questions and users without performance degradation.
2. **Performance:** Page load times should be under 2 seconds, utilizing caching mechanisms.
3. **Security:** Protection against SQL Injection (via prepared statements) and Cross-Site Scripting (XSS).

4. **Availability:** The system should be available 24/7 with minimal downtime.

6.4 System Architecture

The system follows the **LAMP** architecture (Linux, Apache, MySQL, PHP), utilizing WordPress as the framework.

- **Presentation Layer (Frontend):**
 - Built using HTML5, CSS3, and JavaScript.
 - Responsible for the User Interface (UI), displaying the quiz forms, and handling client-side events like the timer.
- **Application Layer (Backend):**
 - Powered by PHP (WordPress Core).
 - Handles the business logic: processing form submissions, validating answers, and managing user sessions.
- **Data Layer (Database):**
 - Uses **MySQL** (Relational Database).
 - Stores structured data including User Credentials (`wp_users`), Quiz Questions (`wp_posts`), and Leaderboard Scores (Custom Table).

6.5 Data Flow Diagram (DFD) Description

1. **Input:** The user selects an answer and clicks "Submit" (or the timer runs out).
2. **Processing:**
 - JavaScript captures the input and sends an AJAX request to the server.
 - PHP receives the request, verifies the **Nonce** (security token), and compares the user's answer with the correct answer stored in the database.

- PHP calculates the score and updates the user's total in the database.
3. **Output:** The server returns a JSON response containing the score and the correct answers. The browser updates the UI to show the "Results Card" and updates the Leaderboard.
-

6. SYSTEM REQUIREMENTS

7.1 Hardware Requirements (Development & Server)

To develop and host "Quiz by Shivam" efficiently, the following hardware specifications are required for the development machine and the hosting server:

- **Processor (CPU):** Intel Core i3 (Gen 5 or newer) or AMD Ryzen 3 equivalent. WordPress local development environments (like XAMPP) require moderate processing power.
- **Memory (RAM):**
 - **Minimum:** 4 GB RAM.
 - **Recommended:** 8 GB RAM to run local servers (Apache/MySQL), web browsers, and code editors simultaneously without lag.
- **Storage:** 256 GB SSD (Solid State Drive). SSDs are highly recommended over HDDs for faster database read/write operations during development.
- **Internet Connection:** High-speed broadband is required for installing plugins, updating WordPress core, and managing remote database connections.

7.2 Software Requirements (Development Environment)

The following software tools were used to build, test, and deploy the application:

- **Operating System:** Windows 10/11, macOS, or Linux (Ubuntu 20.04+).
- **Local Server Environment:** XAMPP, WAMP, or LocalWP (required to run Apache and MySQL locally).
- **Backend Technology:** PHP 7.4 or PHP 8.x (WordPress Core).
- **Database Management:** MySQL (via phpMyAdmin) for creating tables and managing data.

- **Code Editor:** Visual Studio Code (VS Code) equipped with extensions like *PHP Intelephense* and *WordPress Snippets*.
- **Browser:** Google Chrome or Mozilla Firefox (with Developer Tools for debugging CSS and JavaScript).
- **Version Control:** Git and GitHub for source code management.

7.3 Client-Side Requirements (User)

Since "Quiz by Shivam" is a web-based application, users do not need to install specific software. They can access the platform using:

A. On Desktops/Laptops:

- **Web Browser:** Any modern browser supporting HTML5 and ES6 JavaScript (Chrome, Edge, Firefox, Safari).
- **RAM:** 4 GB or higher for smooth rendering of animations.

B. On Mobile Devices:

- **Processor:** Snapdragon 600 series / Apple A10 or equivalent.
- **OS:** Android 10+ or iOS 14+.
- **Screen Resolution:** Minimum 720p HD (The responsive design automatically adapts layout).

7. SYSTEM DESIGN AND CODING

8.1 Introduction to System Design

Software design is a critical phase in the development lifecycle of "Quiz by Shivam." It establishes the architectural foundation for how the WordPress components (Themes, Plugins, Database) interact.

Design is initiated after the system requirements—such as the need for categorized quizzes, timed assessments, and leaderboards—have been analyzed.

The primary goal of this phase is to create a scalable model of the quiz platform that ensures seamless data flow between the Client (Browser) and the Server (WordPress/PHP). The design focuses on **User Experience (UX)** and **Data Integrity**. By translating requirements into a structured system, we minimize the risk of bugs and ensure secure data handling (e.g., using WordPress Nonces for security).

In this phase, detailed representations of data structures (MySQL table schemas), program logic (Quiz Timer and Scoring Algorithms), and procedural steps are defined.

8.2 Input Design

Input design is the process of converting user-oriented inputs into a computer-readable format. For "Quiz by Shivam," this involves designing the forms and interactive elements users engage with.

Primary Objectives:

1. **Efficiency:** To minimize user effort (e.g., using radio buttons for quiz answers instead of typing).
2. **Accuracy:** To ensure high data integrity by implementing validation rules (e.g., preventing a user from registering without a valid email).

3. **Usability:** To ensure complex actions, such as submitting a timed quiz, are intuitive.

Key Input Interfaces Designed:

- **User Registration Form:** A secure input interface for Email, Username, and Password. It includes real-time validation to check if a username already exists.
- **Quiz Interface:** A dynamic form displaying the question and four options (Radio Buttons). It captures the user's selection and locks the input once the timer hits zero.
- **Admin Panel Inputs:** A backend interface allowing administrators to input Question Text, Options, Correct Answer, and Category using standard WordPress text fields and dropdowns.
- **Search & Filter:** A search bar input that allows users to filter quizzes by category (e.g., "DBMS", "Networking").

Data Verification Strategy:

- **Client-Side:** JavaScript checks if all required fields are filled before the "Submit" button becomes active.
- **Server-Side:** PHP sanitizes all inputs using `sanitize_text_field()` to prevent SQL Injection and Cross-Site Scripting (XSS) attacks.

8.3 Output Design

Output design refers to how the results of processing are communicated to the users. In this project, output design is crucial for displaying Quiz Results and Leaderboard Rankings in a motivating way.

Categories of System Outputs:

1. **Interactive Outputs:** The **Quiz Result Card** is the primary output. Once a user submits a quiz, the system calculates the score and instantly displays the percentage, correct answers, and a performance message (e.g., "Excellent Work!").
2. **Visual Outputs (Leaderboard):** The Global Leaderboard is a visual representation of data. It outputs a ranked table showing "Rank," "User Name," and "Total Score," sorted dynamically by highest points.
3. **Operational Outputs:** System feedback messages such as "Registration Successful" or "Incorrect Password" help users understand the system status.
4. **Turnaround Outputs:** The "Edit Profile" screen outputs existing user data (User Bio, Avatar) so the user can modify it.

Key Output Design Principles:

- **Responsiveness:** Using the **Bootstrap** grid system, outputs adapt to screen size. A leaderboard table that is wide on a desktop will scroll horizontally or stack vertically on a mobile device.
- **Hierarchical Structure:** Information is organized logically. For example, a Quiz Card displays the *Category Name* first, followed by the *Quiz Title*, and finally the *Start Button*.
- **Clarity:** Scores are color-coded (Green for high scores, Red for low scores) to provide immediate visual cues to the student.

8.4 Conclusion

In the software engineering process of "Quiz by Shivam," system design proved foundational for building a reliable academic platform. The rigorous **Input Design** ensures that student data and quiz answers are captured accurately and validated before storage. Meanwhile, the thoughtful **Output Design** guarantees that knowledge is presented in a well-structured, visually appealing manner. By

leveraging the **WordPress (PHP/MySQL)** stack, the system meets the needs of the modern student community while remaining scalable for future enhancements.

8. DATABASE DESIGN

1. Quiz Answers:

Column	Type	Comment
id	int(150) unsigned <i>Auto Increment</i>	
question_id	int(11) unsigned	
answer	text	
image	text <i>NULL</i>	
correct	tinyint(1)	
ordering	int(11)	
weight	double <i>NULL</i> [0]	
placeholder	text <i>NULL</i>	

2.Quiz Categories:

Column	Type	Comment
id	int(16) unsigned <i>Auto Increment</i>	
author_id	int(16) unsigned [0]	
title	varchar(256)	
description	text	
published	tinyint(3) unsigned	

3.Quiz terms:

Column	Type	Comment
id	bigint(20) <i>Auto Increment</i>	
question_id	int(11) <i>NULL</i> [0]	
quiz_id	int(11) <i>NULL</i> [0]	
term_id	int(11) <i>NULL</i> [0]	
taxonomy	varchar(50) <i>NULL</i>	

4.Quiz Questions:

Column	Type	Comment
id	int(16) unsigned <i>Auto Increment</i>	
author_id	int(16) unsigned [0]	
category_id	int(16) unsigned	
question	text	
question_title	text	
question_image	text <i>NULL</i>	
wrong_answer_text	text <i>NULL</i>	
right_answer_text	text <i>NULL</i>	
question_hint	text <i>NULL</i>	
explanation	text <i>NULL</i>	
type	varchar(256)	
published	tinyint(3) unsigned	
create_date	datetime <i>NULL</i>	
not_influence_to_score	text <i>NULL</i>	
weight	double <i>NULL</i> [1]	
options	text <i>NULL</i>	

5.Quiz Reports:

Column	Type	Comment
id	int(11) <i>Auto Increment</i>	
question_id	int(11)	
report_text	text	
resolved	tinyint(1) [0]	
create_date	datetime <i>NULL</i>	
resolve_date	datetime <i>NULL</i>	
user_id	int(16) unsigned <i>NULL</i>	
user_name	varchar(256) <i>NULL</i>	
user_email	varchar(256) <i>NULL</i>	

6.Quiz :

Column	Type	Comment
id	int(16) unsigned <i>Auto Increment</i>	
author_id	int(16) unsigned [0]	
title	varchar(256)	
description	text	
quiz_image	text <i>NULL</i>	
quiz_category_id	int(11) unsigned	
question_ids	text	
ordering	int(16)	
quiz_url	text <i>NULL</i>	
published	tinyint(3) unsigned	
create_date	datetime <i>NULL</i>	
custom_post_id	int(16) unsigned <i>NULL</i>	
options	text <i>NULL</i>	
intervals	text <i>NULL</i>	

7.Quiz Rates:

Column	Type	Comment
id	int(11) <i>Auto Increment</i>	
quiz_id	int(11)	
user_id	int(11)	
user_ip	varchar(256)	
user_name	text <i>NULL</i>	
user_email	text <i>NULL</i>	
user_phone	text <i>NULL</i>	
rate_date	datetime	
score	varchar(256)	
review	text	
options	text	

9. DATA FLOW DIAGRAM (DFD)

A Data Flow Diagram (DFD) is a graphical tool used to describe and analyze the flow of data through a system. It focuses on the data flowing into the Quiz by Shivam platform, the processes that transform the data (quiz evaluation, score storage, leaderboard ranking), and the data flowing out to storage or to the user.

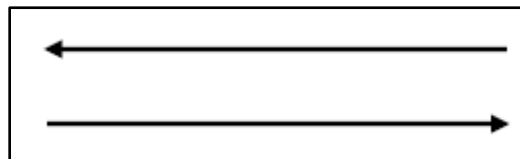
DFDs are of two types:

1. **Physical DFD:** The physical DFD is a model of the current system and is used to ensure that the current system has been clearly understood.
2. **Logical DFD:** The logical DFD is a model of the proposed system. It clearly shows the requirements on which the new system should be built, focusing on *what* happens, not *how* it

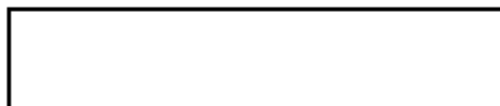
Notation Used In DFD:

There are four simple notations used to complete DFDs:

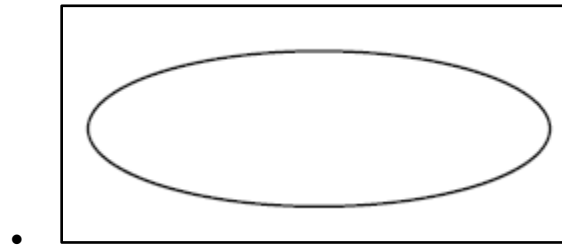
- **Dataflow:**



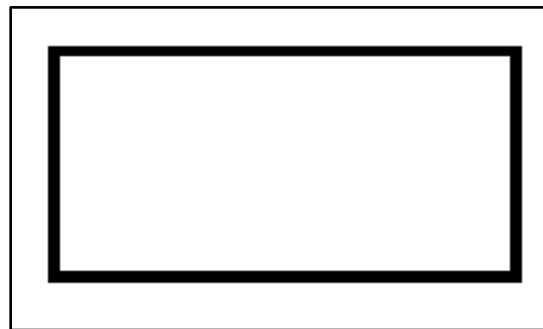
- **External Entity:**



- **Process:**

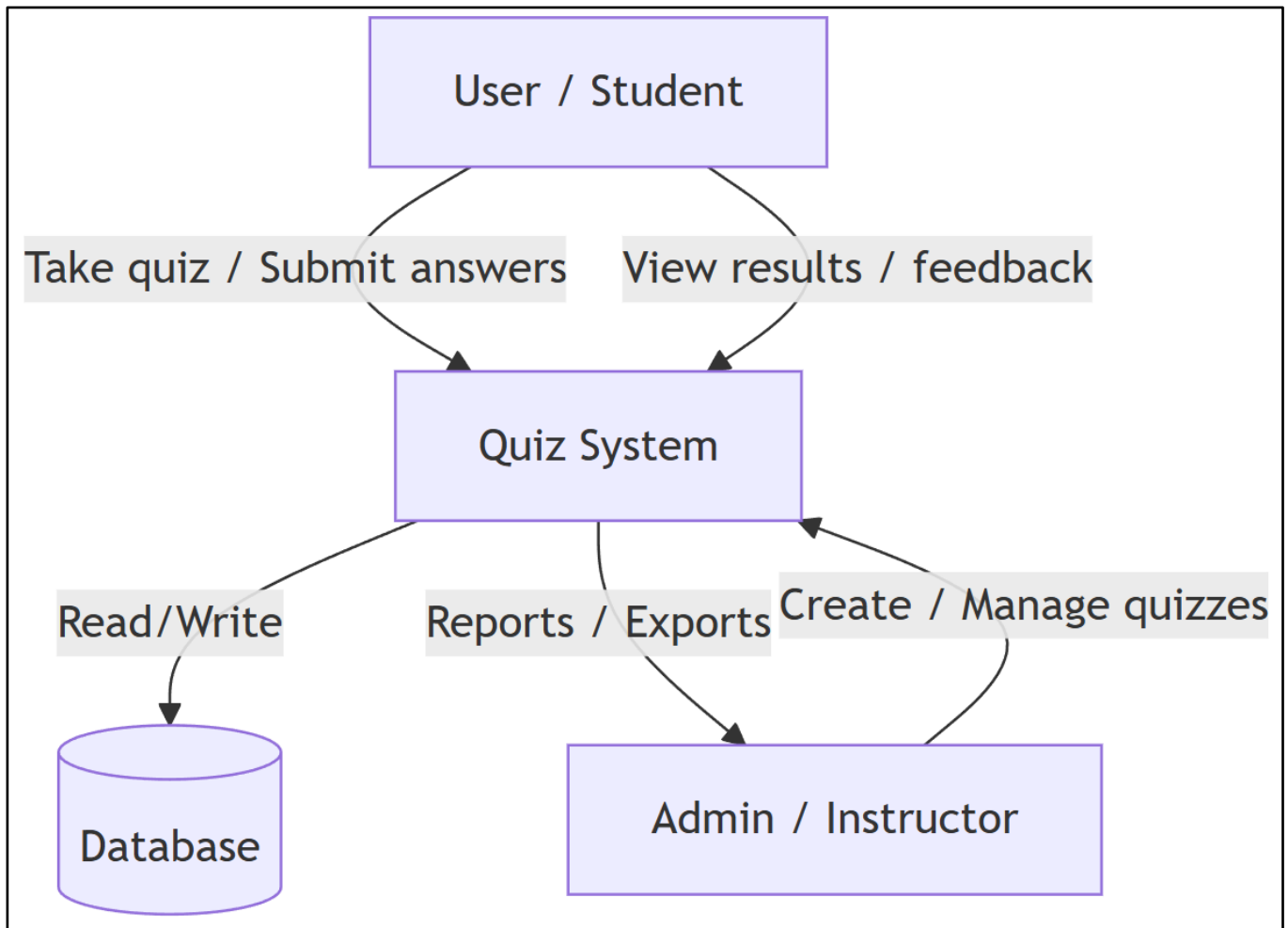


- **Data store:**

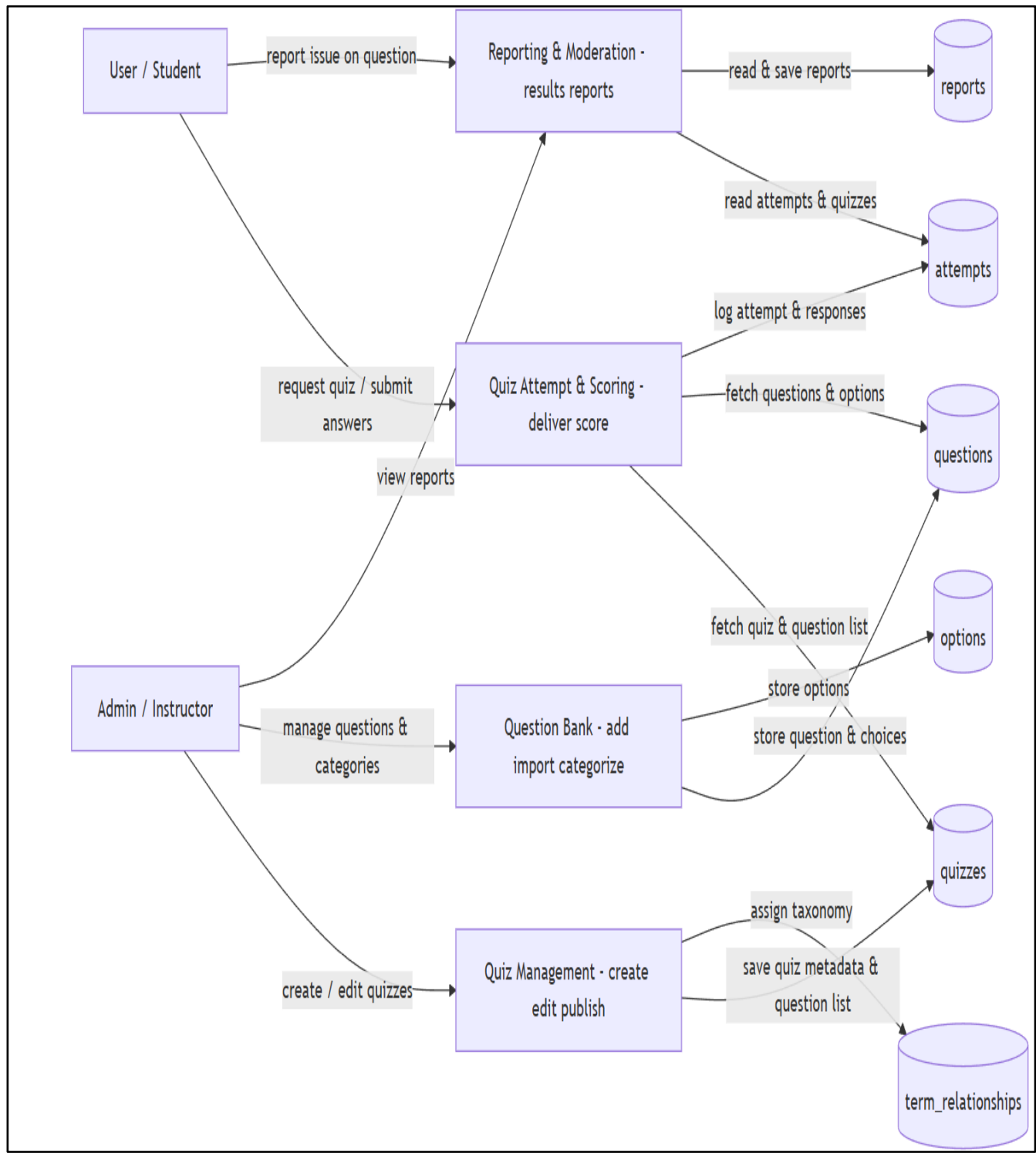


DFD FOR THE SYSTEM

2. Level 0:



3. Level 1:



10. ENTITY RELATIONSHIP DIAGRAM

An Entity Relationship Diagram (ERD) expresses the overall logical structure of the database graphically. It illustrates the interrelationships between the entities in the Quiz by Shivam system, providing a clear blueprint of how data is organized and connected within the MySQL database.

The components of the E-R Diagram are:

1. **Entity:** An entity is a real-world object or concept that is distinguishable from all other objects. In this project, examples include User, Quiz Category, and Quiz Question.
2. **Relationship:** It is an association among several entities. For instance, a "User" creates a "Quiz," representing a relationship between the two.
3. **Weak Entity:** A weak entity is an entity that cannot be uniquely identified by its own attributes alone and depends on a strong entity. In this system, a Quiz Attempt can be viewed conceptually as a weak entity because it is dependent on the existence of a specific User and Quiz Category.
4. **Attributes:** A property or characteristic of an entity. For example, a User entity has attributes like Username, Email, and Password.

ENTITIES IN QUIZ BY SHIVAM

Based on the MySQL table schema designed for the system, the key entities and their attributes are:

1. **User:** Represents the registered quiz participants on the platform.
 - **Attributes:** _id, firstName, lastName, username, email, password, bio, avatar, isVerified, followers, following.

2. Quiz Category: Represents the quiz subjects available on the platform.
 - Attributes: `_id`, `title`, `slug`, `content`, `space`, `tags`, `viewsCount`, `scores`, `ranks`, `quiz_data`, `createdAt`.
 3. Quiz Question: Represents individual questions within a quiz category.
 - Attributes: `_id`, `content`, `parentQuiz`, `parentComment`, `scores`, `ranks`, `createdAt`.
 4. Quiz Attempt: Represents a single quiz session completed by a user.
 - Attributes: `_id`, `name`, `description`, `avatar`.
 5. Leaderboard Entry: Represents a user's ranked position based on their total score across all categories.
 - Attributes: `_id`, `name`, `createdBy`.
-

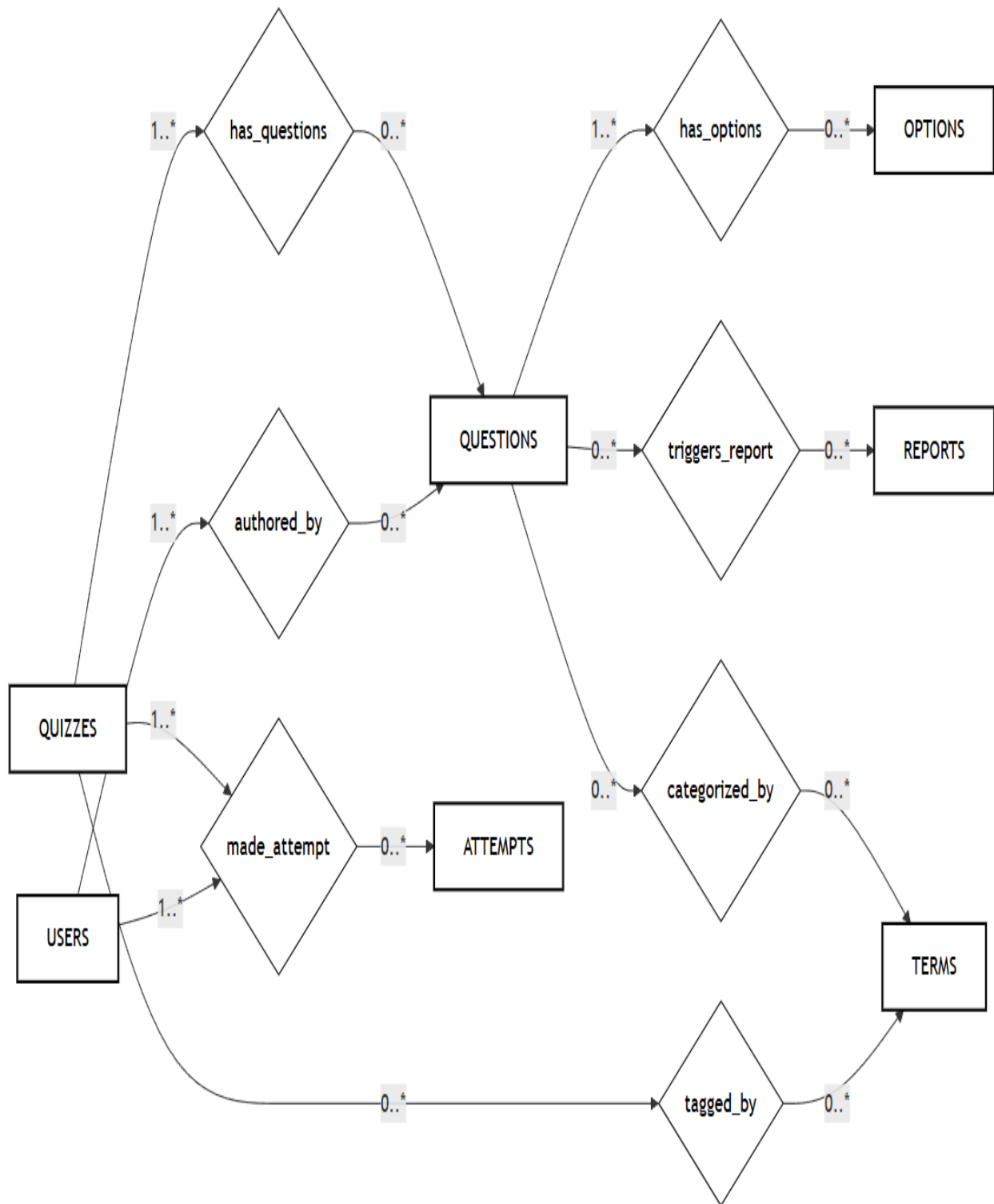
RELATIONSHIPS

The system enforces the following relationships to ensure data integrity and connectivity:

1. User — Quiz (1:N): A single User can create multiple Quizzes (One-to-Many).
2. User — Comment (1:N): A single User can post multiple Comments across different threads.
3. Quiz — Comment (1:N): A single Quiz can have many Comments associated with it.
4. Space — Quiz (1:N): A Space (Category) acts as a container for multiple Quizzes.

5. Quiz — Tag (M:N): A Quiz can have multiple Tags, and a Tag can be associated with multiple Quizzes (Many-to-Many).
 6. User — User (M:N): A User can follow many other Users, and can be followed by many Users (Recursive Many-to-Many).
-

ENTITY RELATIONSHIP DIAGRAM



11. INPUT FORMS, PAGES AND CODING

Home Page:

QUIZ by Shivam
Test Your Knowledge

[Home](#) [Account](#) [Logout](#) [Register](#)



Click on any category, take a **quiz**, and see where you rank!

Start your journey by selecting a category that interests you, putting your skills to the test, and unlocking your official rank on the global leaderboard.

[Get Start](#)



explore courses



Database Management System

Database Management Systems (DBMS)



Web Technologies



Operating Systems



PROGRAMMING

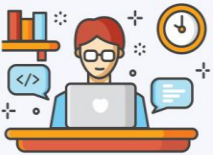
Programming in C



Artificial Intelligence



Computer Network



Software Engineering



Computer Fundamental

Register Page:

QUIZ by Shivam
Test Your Knowledge

[Home](#) [Register](#) [Login](#)



Register

Username

shivamadmin

First Name

Last Name

E-mail Address

Password

***** 

Confirm Password

Confirm Password 

Register

Login

Login Page:

QUIZ by Shivam
Test Your Knowledge

[Home](#) [Register](#) [Login](#)



Login

Username or E-mail

shivamadmin

Password

👁

☐ Keep me signed in

Login

Register

[Forgot your password?](#)

Get free Domain
(yourname.24x7ithub.com)
Plan only at just 2500/-

Website hosting
for
school/collage .

[Home](#) [Register](#) [Login](#)

Copyright © 2026 QUIZ by Shivam | Powered by QUIZ by Shivam.

Quiz Page:

QUIZ by Shivam
Test Your Knowledge

[Home](#) [Account](#) [Logout](#) [Register](#)



Database Management Systems (DBMS)

Leave a Comment / By shivam / December 22, 2025

Which level describes how data is stored?

☐ External
☐ Conceptual
☐ Internal
☐ Logical

Which command removes all rows but keeps structure?

☐ DELETE
☐ DROP
☐ TRUNCATE
☐ REMOVE

Previous

20 %

Next

[← Previous Quiz](#)[Next Quiz →](#)

Leave a Reply

Logged in as Shivam Singh. [Edit your profile.](#) Log out? Required fields are marked *

Comment *

Post Comment

40

Quiz Result Page:

Web Technologies

Leave a Comment / By shivam / December 25, 2025

Thanks for submitting your response! You can edit this message on the "Results Pages" tab.

Which technology stores cookies?

- ☐ HTML
- ☒ Browser
- ☐ Server
- ☒ Database

Which file extension for JavaScript?

- ☐ .java
- ☒ .js
- ☐ .jsp
- ☐ .json

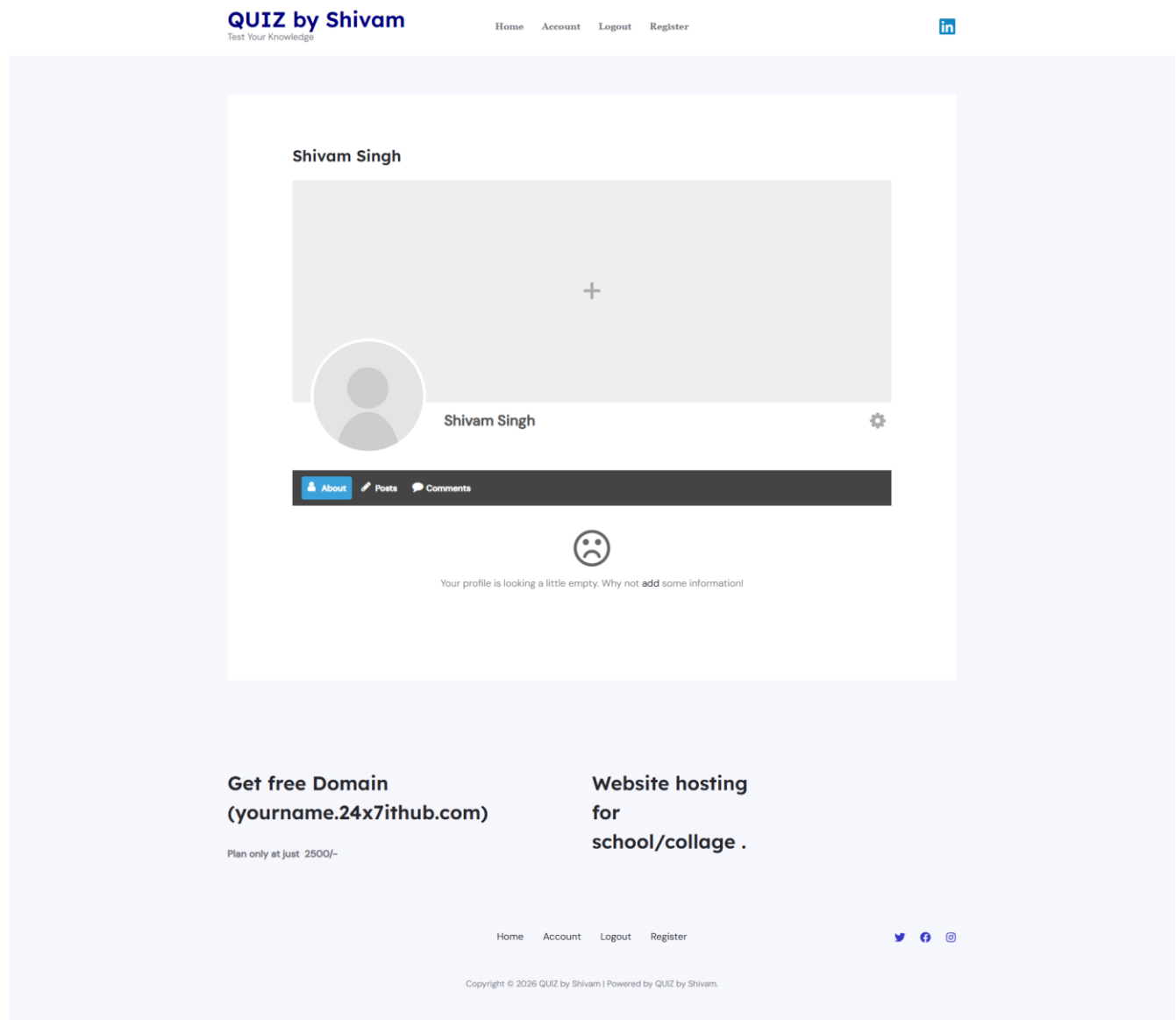
Which tag defines table row?

- ☒ td
- ☐ th
- ☒ tr
- ☐ table

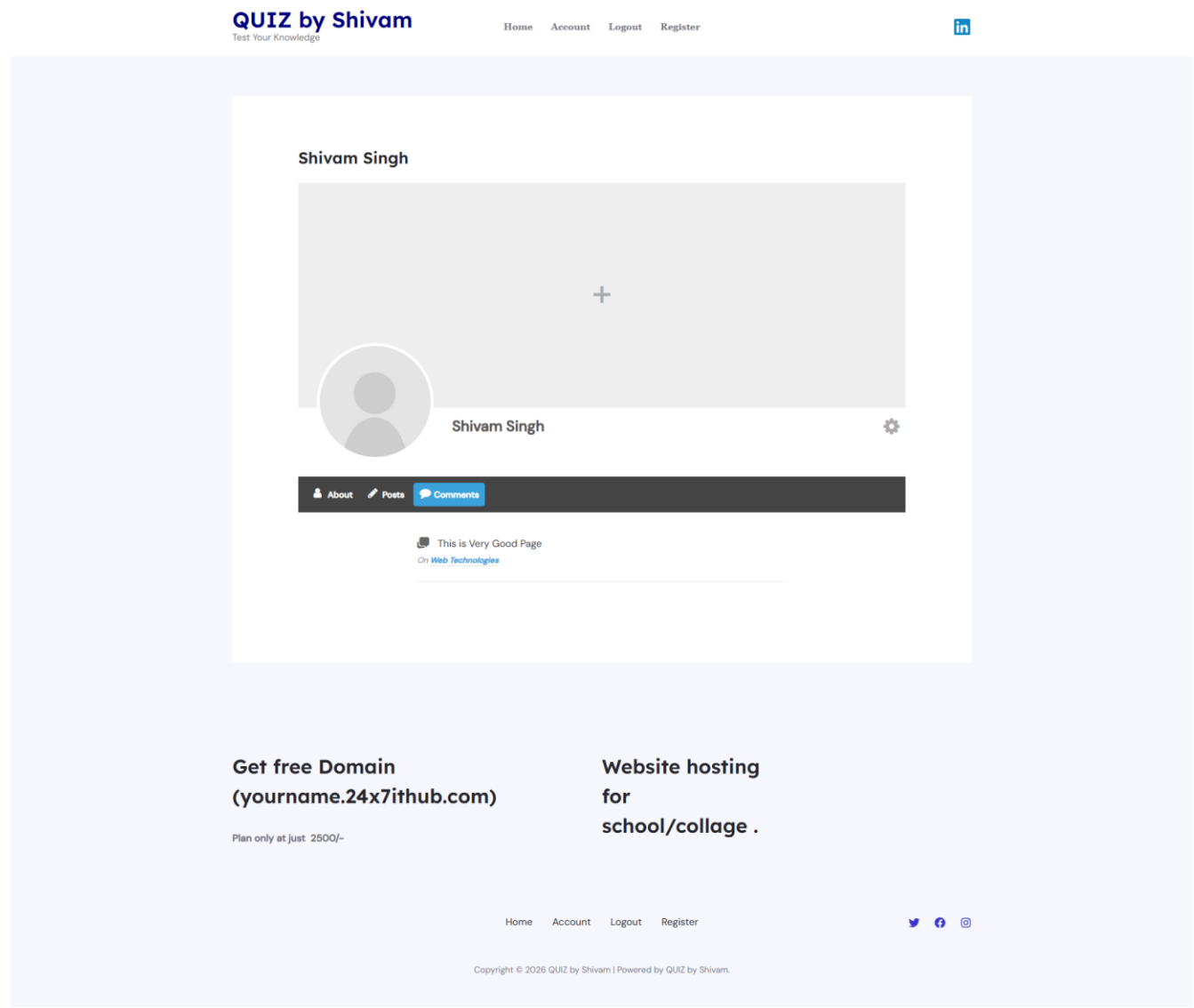
JavaScript is a:

- ☒ Markup language
- ☐ Style sheet
- ☒ Scripting language
- ☐ Database

User About Page:



User Comment Page:



LIMITATIONS

While **Quiz by Shivam** successfully achieves its primary objectives of fostering a collaborative academic environment, there are certain limitations in the current version of the system:

1. **Lack of Multi-Language Support:** The platform currently operates solely in English. Given the diverse linguistic background of students in many colleges, this limits accessibility for those who might prefer discussing complex academic concepts in their native regional languages (e.g., Hindi, Marathi).
2. **Absence of Private Messaging (Direct Chat):** The system currently focuses on public discussions via Quizzes and Comments. It does not yet offer a real-time private messaging (DM) feature. Students wishing to discuss sensitive matters or collaborate one-on-one must rely on external communication tools, which breaks the continuity of the platform experience.
3. **Basic Content Recommendation Algorithm:** The current "Popular" and "Latest" feeds rely on simple sorting logic based on timestamps and view counts. The platform lacks an intelligent, AI-driven recommendation engine that could suggest quizzes based on a student's specific reading history, joined quiz categories, or past upvoting behavior.
4. **Limited File Sharing Capabilities:** While users can upload images (avatars) via Cloudinary, the platform does not yet support the direct uploading and sharing of heavy academic documents, such as PDF notes, past year question papers, or project zip files, which are essential for a complete academic resource hub.

FUTURE SCOPE OF QUIZ BY SHIVAM

The current implementation of the Quiz by Shivam serves as a robust foundation. However, there is significant potential for expansion and enhancement in future iterations:

1. **Real-Time Chat & Collaboration:** Integrating **Socket.io** to enable real-time features such as private messaging between students and group chats within specific "Spaces." This would allow for instant study group coordination without leaving the platform.
2. **Mobile Application Development:** Developing a native mobile application using **React Native**. Since the backend is already built as a RESTful API (WordPress/PHP/Express), it can easily serve a mobile app, providing students with push notifications for exam alerts, replies, and trending quizzes directly on their smartphones.
3. **AI-Powered Content Moderation & Tagging:** Integrating AI libraries (like TensorFlow.js) to automatically detect and flag inappropriate content or spam. Additionally, AI could be used to auto-suggest tags for new quizzes based on the content text, ensuring better categorization and searchability.
4. **Academic Resource Repository:** Expanding the quiz categories feature to include a dedicated **Document Repository**. This would allow students to upload, organize, and download lecture notes, eBooks, and research papers, turning the quiz platform into a complete Learning Management System (LMS) component.
5. **Gamification and Reputation System:** Enhancing the current "Top Contributors" feature by introducing badges (e.g., "Problem Solver," "Verified Tutor") and a reputation point system. This would incentivize students to provide high-quality answers and remain active on the platform.

CONCLUSION

The **Quiz by Shivam** project successfully addresses the critical need for a centralized, structured, and interactive digital community for students. By bridging the gap between social networking and academic collaboration, the platform provides a dedicated space where knowledge can be shared, validated, and preserved.

The development process demonstrated the power and flexibility of the **LAMP) stack** (MySQL, WordPress PHP, WordPress themes with custom HTML/CSS/JavaScript, WordPress/PHP). The use of **WordPress** ensured a dynamic, single-page user interface that is responsive across devices, while **WordPress** provided efficient state management for complex features like real-time quizing and authentication. On the backend, **Server-side PHP** and **MySQL** proved to be a highly scalable combination, capable of handling diverse data types—from unstructured discussion text to structured user profiles.

Key features such as **Spaces** for categorization, **Competitive Features** for community opinion, and the **Upvote/Downvote** mechanism for quality control have created a democratized environment where the best content naturally rises to the top. The implementation of secure authentication using **WordPress nonces** and **WordPress password hashing** ensures that the platform remains safe and trustworthy for all students.

In conclusion, Quiz by Shivam is not just a discussion board; it is a scalable, modern solution for campus interaction. With its solid architectural foundation and clear path for future upgrades, it is poised to become an indispensable tool for student success and peer-to-peer learning.

BIBLIOGRAPHY

The following resources, documentation, and literature were referred to during the design, development, and implementation of this project:

Books:

1. *Pro LAMP) stack: Full Stack Web App Development with Mongo, Express, React, and Node* by Matthew MacDonald.
2. *JavaScript: The Good Parts* by Douglas Crockford.
3. *Software Engineering: A Practitioner's Approach* by Roger S. Pressman.

Official Documentation & Websites:

4. **WordPress themes with custom HTML/CSS/JavaScript Documentation** - <https://react.dev>
5. **WordPress/PHP Documentation** - <https://nodejs.org/en/docs>
6. **MySQL Manual** - <https://www.mongodb.com/docs>
7. **WordPress state management** - <https://redux-toolkit.js.org>
8. **Bootstrap (WordPress)** - <https://react-bootstrap.github.io>
9. **MDN Web Docs (Mozilla)** - <https://developer.mozilla.org>

Tools & Libraries:

10. **Cloudinary (Image Management)** - <https://cloudinary.com>
11. **WordPress nonces (JSON Web Tokens)** - <https://jwt.io>

12. **Axios (HTTP Client)** - <https://axios-http.com>

13. **React Icons** - <https://react-icons.github.io/react-icons>

14. **Draw.io (System Diagrams)** - <https://app.diagrams.net>